STAR RX72N - blinky Bench Test

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

Chapter 3

Directory Documentation

3.1 blinky Directory Reference

Files

- file linker.ld
- file main.c

  RX72N multi-LED breathing cycle test: PA7, PB0, P71, P72, PB1, PB2.

Chapter 4

File Documentation

4.1 blinky/linker.ld File Reference

4.2 linker.ld

Go to the documentation of this file.

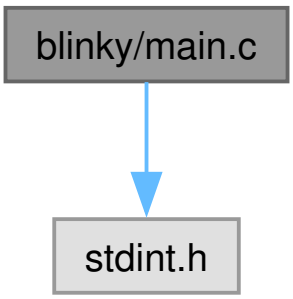
```
00001 /*
00002  * blinky/linker.ld -- Minimal linker script for RX72N bare-metal blinky.
00003  *
00004  * RX72N memory map (R5F572NNHxFB, 144-pin):
00005  *   Internal RAM : 0x00000000 - 0x0007FFFF  (512 KB)
00006  *   Internal ROM : 0xFFE00000 - 0xFFFFFFFF  (2 MB active region)
00007  *
00008  * Note: RAM starts at 0x4 not 0x0. The first 4 bytes (0x0-0x3) are reserved
00009  * by the hardware (used as the undefined instruction trap vector on RX).
00010  *
00011  * Fixed hardware vector locations (RX CPU specification):
00012  *   0xFFFFFF80 - 0xFFFFFFFC : Exception/interrupt vector table
00013  *   0xFFFFFFFC - 0xFFFFFFFF : Reset vector (4 bytes, points to startup code)
00014  */
00015
00016 MEMORY
00017 {
00018     RAM : ORIGIN = 0x00000004, LENGTH = 0x7FFFC    /* 512 KB - 4 bytes */
00019     ROM : ORIGIN = 0xFFE00000, LENGTH = 0x200000   /* 2 MB             */
00020 }
00021
00022 SECTIONS
00023 {
00024     /*
00025      * Fixed reset vector at the very top of flash.
00026      * The CPU reads this 32-bit address on power-on and jumps there.
00027      */
00028     .fvectors 0xFFFFFFFC : AT(0xFFFFFFFC)
00029     {
00030         KEEP(*(.fvectors))
00031     } > ROM
00032
00033     /* Code and read-only data starting at base of ROM */
00034     .text 0xFFE00000 : AT(0xFFE00000)
00035     {
00036         *(.text*)
00037         *(.rodata*)
00038         etext = .;
00039     } > ROM
00040
00041     /* Uninitialised data (BSS) -- main.c has no globals, so this is empty */
00042     .bss :
00043     {
00044         _bss_start = .;
00045         *(.bss*)
00046         *(COMMON)
00047         _bss_end = .;
00048     } > RAM
00049
```

```
00050     /*
00051      * Stack definitions.
00052      * USP (user stack): top of RAM, grows downward.
00053      * ISP (interrupt stack): 2 KB below USP, grows downward.
00054      */
00055     _ustack = ORIGIN(RAM) + LENGTH(RAM);
00056     _istack = _ustack - 0x800;
00057 }
```

4.3 blinky/main.c File Reference

RX72N multi-LED breathing cycle test: PA7, PB0, P71, P72, PB1, PB2.

#include <stdint.h>
Include dependency graph for main.c:



Data Structures

- struct led_t

  Binds a GPIO output register to a single pin's bit mask.

Enumerations

- enum port_addr_t : uintptr_t { k_port7_pdr_addr = 0x0008C007U , k_port7_podr_addr = 0x0008C027U , k_porta_pdr_addr = 0x0008C00AU , k_porta_podr_addr = 0x0008C02AU , k_portb_pdr_addr = 0x0008C00BU , k_portb_podr_addr = 0x0008C02BU }
- enum porta_pins_t : uint8_t { k_pin_pa7 = (uint8_t)(1U << 7U) }
- enum port7_pins_t : uint8_t { k_pin_p71 = (uint8_t)(1U << 1U) , k_pin_p72 = (uint8_t)(1U << 2U) }
- enum portb_pins_t : uint8_t { k_pin_pb0 = (uint8_t)(1U << 0U) , k_pin_pb1 = (uint8_t)(1U << 1U) , k_pin_pb2 = (uint8_t)(1U << 2U) }
- enum breathe_params_t : uint32_t { k_max_bright = 50U , k_reps_per_step = 5U , k_flash_count = 3U , k_flash_on_cnt = 8000U , k_flash_off_cnt = 8000U }
- enum led_count_t : uint8_t { k_num_leds = 6U }

Functions

- static volatile uint8_t * port7_pdr (void)
- static volatile uint8_t * port7_podr (void)
- static volatile uint8_t * porta_pdr (void)
- static volatile uint8_t * porta_podr (void)
- static volatile uint8_t * portb_pdr (void)
- static volatile uint8_t * portb_podr (void)
- static void delay (uint32_t count)

  Spin for approximately count loop iterations.
- static void led_on (const led_t *led)
- static void led_off (const led_t *led)
- static void breathe_ramp (const led_t *led, uint32_t invert)

  Run one PWM ramp pass on led.
- static void breathe (const led_t *led)

  Run one full breathe cycle (fade in then fade out) on led.
- static void flash_all (const led_t *leds, uint8_t count)

  Flash all LEDs k_flash_count times to signal cycle completion.
- int main (void)

  Configure GPIO outputs and run the breathing cycle forever.

4.3.1 Detailed Description

RX72N multi-LED breathing cycle test: PA7, PB0, P71, P72, PB1, PB2.

Bare-metal test with no RTOS, no BSP, and no external clock configuration. Six LEDs breathe sequentially using software PWM, then all flash together to signal the end of one full cycle before repeating.

Breathing sequence: PA7 -> PB0 -> P71 -> P72 -> PB1 -> PB2 -> [flash all x3] -> repeat

Runs on the default internal LOCO clock at 240 kHz (OFS1 = 0xFFFFFFFF). Software PWM timing is calibrated for this clock.

Port register addresses from RX72N Hardware Manual (r01uh0824ej0111): PDR = 0x0008C000 + port_index (direction: 0=input, 1=output) PODR = 0x0008C020 + port_index (output data) Port indices: 7=0x07, A=0x0A, B=0x0B

After reset, PMR registers default to 0x00 (all GPIO) so no MPC unlock is required.

Built with -O0 so volatile delay loops are not optimised out.

SPDX-License-Identifier: MIT

Copyright

Definition in file main.c.

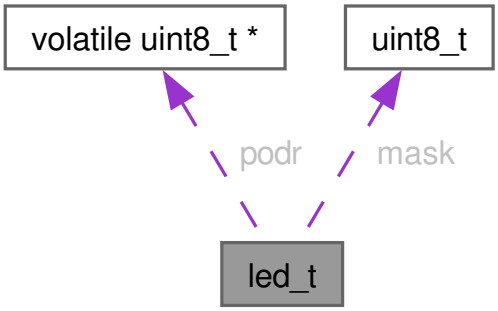
4.3.2 Data Structure Documentation

4.3.2.1 struct led_t

Binds a GPIO output register to a single pin's bit mask.

Definition at line 97 of file main.c.

Collaboration diagram for led_t:



Data Fields

uint8_t	mask	Bit mask selecting this LED's pin
volatile uint8_t *	podr	Port output data register for this LED

4.3.3 Enumeration Type Documentation

4.3.3.1 breathe_params_t

enum breathe_params_t : uint32_t

Enumerator

k_max_bright	PWM duty-cycle upper bound (steps 0..50)
k_reps_per_step	PWM cycles held at each brightness step
k_flash_count	Completion flashes after one full cycle
k_flash_on_cnt	Flash ON busy-wait count
k_flash_off_cnt	Flash OFF busy-wait count

Definition at line 76 of file main.c.

```
00076             : uint32_t {
00077     k_max_bright     = 50U,   /**< PWM duty-cycle upper bound (steps 0..50) */
00078     k_reps_per_step  = 5U,    /**< PWM cycles held at each brightness step */
00079     k_flash_count    = 3U,    /**< Completion flashes after one full cycle */
00080     k_flash_on_cnt   = 8000U, /**< Flash ON busy-wait count */
00081     k_flash_off_cnt  = 8000U, /**< Flash OFF busy-wait count */
00082 } breathe_params_t;
```

4.3.3.2 led_count_t

enum led_count_t : uint8_t

Enumerator

k_num_leds	LEDs in the breathing sequence

Definition at line 87 of file main.c.

```
00087             : uint8_t {
00088     k_num_leds = 6U, /**< LEDs in the breathing sequence */
00089 } led_count_t;
```

4.3.3.3 port7_pins_t

enum port7_pins_t : uint8_t

Enumerator

k_pin_p71	PORT7 pin 1
k_pin_p72	PORT7 pin 2

Definition at line 55 of file main.c.

```
00055             : uint8_t {
00056     k_pin_p71 = (uint8_t)(1U << 1U), /**< PORT7 pin 1 */
00057     k_pin_p72 = (uint8_t)(1U << 2U), /**< PORT7 pin 2 */
00058 } port7_pins_t;
```

4.3.3.4 port_addr_t

enum port_addr_t : uintptr_t

Enumerator

k_port7_pdr_addr	PORT7 direction register
k_port7_podr_addr	PORT7 output data register
k_porta_pdr_addr	PORTA direction register
k_porta_podr_addr	PORTA output data register
k_portb_pdr_addr	PORTB direction register
k_portb_podr_addr	PORTB output data register

Definition at line 39 of file main.c.

```
00039             : uintptr_t {
00040     k_port7_pdr_addr  = 0x0008C007U, /**< PORT7 direction register */
00041     k_port7_podr_addr = 0x0008C027U, /**< PORT7 output data register */
00042     k_porta_pdr_addr  = 0x0008C00AU, /**< PORTA direction register */
00043     k_porta_podr_addr = 0x0008C02AU, /**< PORTA output data register */
00044     k_portb_pdr_addr  = 0x0008C00BU, /**< PORTB direction register */
00045     k_portb_podr_addr = 0x0008C02BU, /**< PORTB output data register */
00046 } port_addr_t;
```

4.3.3.5 porta_pins_t

enum porta_pins_t : uint8_t

Enumerator

k_pin_pa7	PORTA pin 7

Definition at line 51 of file main.c.

```
00051             : uint8_t {
00052     k_pin_pa7 = (uint8_t)(1U << 7U), /**< PORTA pin 7 */
00053 } porta_pins_t;
```

4.3.3.6 portb_pins_t

enum portb_pins_t : uint8_t

Enumerator

k_pin_pb0	PORTB pin 0
k_pin_pb1	PORTB pin 1
k_pin_pb2	PORTB pin 2

Definition at line 60 of file main.c.

```
00060             : uint8_t {
00061     k_pin_pb0 = (uint8_t)(1U << 0U), /**< PORTB pin 0 */
00062     k_pin_pb1 = (uint8_t)(1U << 1U), /**< PORTB pin 1 */
00063     k_pin_pb2 = (uint8_t)(1U << 2U), /**< PORTB pin 2 */
00064 } portb_pins_t;
```

4.3.4 Function Documentation

4.3.4.1 breathe()

void breathe (
const led_t * led) [static]

Run one full breathe cycle (fade in then fade out) on led.

Parameters

in	led	LED to animate.

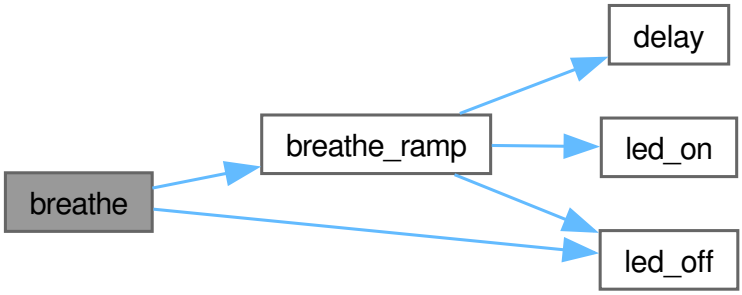
Definition at line 198 of file main.c.

```
00199 {
00200     breathe_ramp(led, 0U); /* in  */
00201     breathe_ramp(led, 1U); /* out */
00202     led_off(led);
00203 }
```

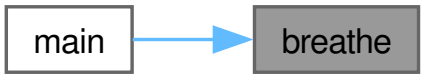
References breathe_ramp(), and led_off().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4.2 breathe_ramp()

```
void breathe_ramp (
        const led_t * led,
        uint32_t invert)    [static]
```

Run one PWM ramp pass on led.

Steps on_time from 0 to k_max_bright (or vice versa depending on invert), holding each level for k_reps_per_step PWM cycles.

Parameters

in	led	LED to animate.
in	invert	When 0, ramp 0->max (fade in). When 1, ramp max->0 (fade out).

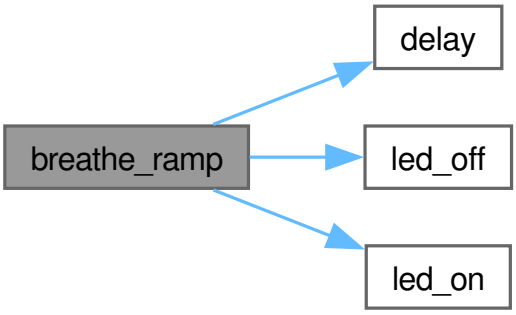
Definition at line 171 of file main.c.

```
00172 {
00173     uint32_t b;
00174     uint32_t rep;
00175     uint32_t on_time;
00176     uint32_t off_time;
00177
00178     for (b = 0U; b <= k_max_bright; b++) {
00179         on_time  = (invert == 0U) ? b : (k_max_bright - b);
00180         off_time = k_max_bright - on_time;
00181         for (rep = 0U; rep < k_reps_per_step; rep++) {
00182             if (on_time > 0U) {
00183                 led_on(led);
00184                 delay(on_time);
00185             }
00186             led_off(led);
00187             if (off_time > 0U) {
00188                 delay(off_time);
00189             }
00190         }
00191     }
00192 }
```

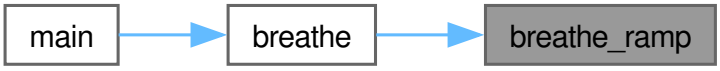
References delay(), k_max_bright, k_reps_per_step, led_off(), and led_on().

Referenced by breathe().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4.3 delay()

void delay (
uint32_t count) [static]

Spin for approximately count loop iterations.

Parameters

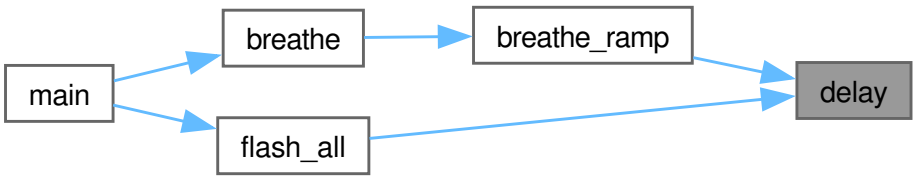
in	count	Iteration count. Each iteration is ~50 us at LOCO 240 kHz.

Definition at line 137 of file main.c.

```
00138 {
00139     volatile uint32_t i;
00140     for (i = 0U; i < count; i++) {
00141         __asm__ __volatile__("nop");
00142     }
00143 }
```

Referenced by breathe_ramp(), and flash_all().

Here is the caller graph for this function:



4.3.4.4 flash_all()

```
void flash_all (
                const led_t * leds,
                uint8_t count)   [static]
```

Flash all LEDs k_flash_count times to signal cycle completion.

Parameters

in	leds	LED descriptor array.
in	count	Number of entries in leds.

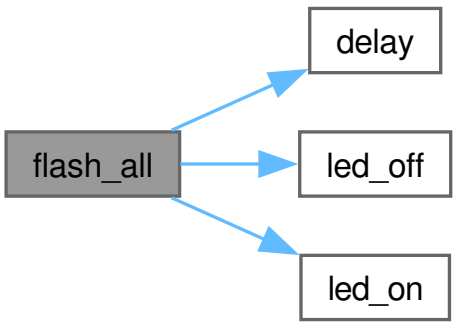
Definition at line 213 of file main.c.

```
00214 {
00215     uint32_t f;
00216     uint8_t  i;
00217
00218     for (f = 0U; f < k_flash_count; f++) {
00219         for (i = 0U; i < count; i++) {
00220             led_on(&leds[i]);
00221         }
00222         delay(k_flash_on_cnt);
00223
00224         for (i = 0U; i < count; i++) {
00225             led_off(&leds[i]);
00226         }
00227         delay(k_flash_off_cnt);
00228     }
00229 }
```

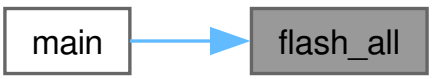
References delay(), k_flash_count, k_flash_off_cnt, k_flash_on_cnt, led_off(), and led_on().

Referenced by main().

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4.5 led_off()

void led_off (
const led_t * led) [inline], [static]

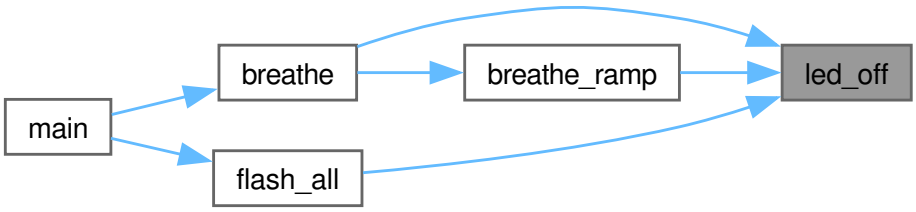
Definition at line 153 of file main.c.

```
00154 {
00155     *led->podr &= (uint8_t)~led->mask;
00156 }
```

References led_t::mask, and led_t::podr.

Referenced by breathe(), breathe_ramp(), and flash_all().

Here is the caller graph for this function:



4.3.4.6 led_on()

void led_on (
const led_t * led) [inline], [static]

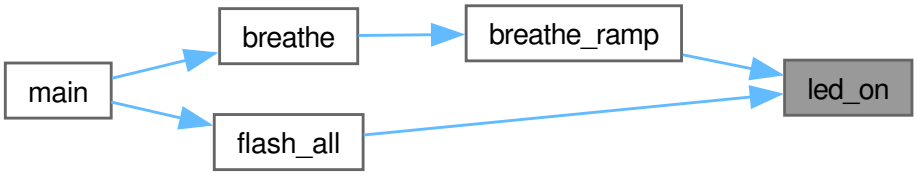
Definition at line 148 of file main.c.

```
00149 {
00150     *led->podr |= led->mask;
00151 }
```

References led_t::mask, and led_t::podr.

Referenced by breathe_ramp(), and flash_all().

Here is the caller graph for this function:



4.3.4.7 main()

```
int main (
                void )
```

Configure GPIO outputs and run the breathing cycle forever.

Returns

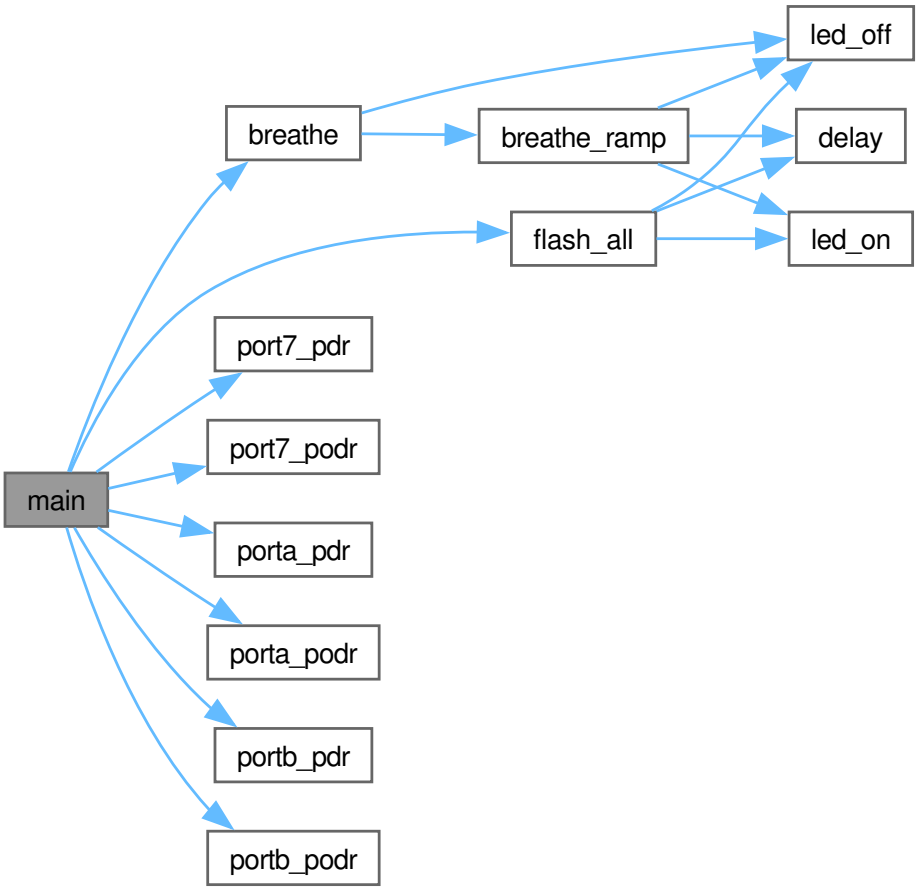
Never returns.

Definition at line 238 of file main.c.

```
00239 {
00240     /* Set LED pins as outputs (PDR bit = 1). */
00241     *porta_pdr() |= (uint8_t)k_pin_pa7;
00242     *port7_pdr() |= (uint8_t)(k_pin_p71 | k_pin_p72);
00243     *portb_pdr() |= (uint8_t)(k_pin_pb0 | k_pin_pb1 | k_pin_pb2);
00244
00245     /* LED table -- order defines the breathing sequence. */
00246     const led_t leds[k_num_leds] = {
00247         { porta_podr(), (uint8_t)k_pin_pa7 },
00248         { portb_podr(), (uint8_t)k_pin_pb0 },
00249         { port7_podr(), (uint8_t)k_pin_p71 },
00250         { port7_podr(), (uint8_t)k_pin_p72 },
00251         { portb_podr(), (uint8_t)k_pin_pb1 },
00252         { portb_podr(), (uint8_t)k_pin_pb2 },
00253     };
00254
00255     for (;;) {
00256         uint8_t i;
00257         for (i = 0U; i < k_num_leds; i++) {
00258             breathe(&leds[i]);
00259         }
00260         flash_all(leds, k_num_leds);
00261     }
00262 }
```

References breathe(), flash_all(), k_num_leds, k_pin_p71, k_pin_p72, k_pin_pa7, k_pin_pb0, k_pin_pb1, k_pin_pb2, port7_pdr(), port7_podr(), porta_pdr(), porta_podr(), portb_pdr(), and portb_podr().

Here is the call graph for this function:



4.3.4.8 port7_pdr()

volatile uint8_t * port7_pdr (
void) [inline], [static]

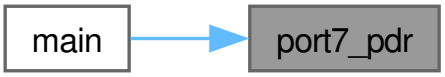
Definition at line 105 of file main.c.

```
00106 {
00107     return (volatile uint8_t *)k_port7_pdr_addr;
00108 }
```

References k_port7_pdr_addr.

Referenced by main().

Here is the caller graph for this function:



4.3.4.9 port7_podr()

```
volatile uint8_t * port7_podr (
                void ) [inline], [static]
```

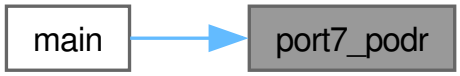
Definition at line 109 of file main.c.

```
00110 {
00111     return (volatile uint8_t *)k_port7_podr_addr;
00112 }
```

References k_port7_podr_addr.

Referenced by main().

Here is the caller graph for this function:



4.3.4.10 porta_pdr()

```
volatile uint8_t * porta_pdr (
                void ) [inline], [static]
```

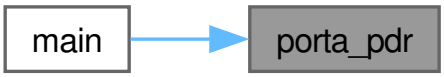
Definition at line 113 of file main.c.

```
00114 {
00115     return (volatile uint8_t *)k_porta_pdr_addr;
00116 }
```

References k_porta_pdr_addr.

Referenced by main().

Here is the caller graph for this function:



4.3.4.11 porta_podr()

```
volatile uint8_t * porta_podr (
                void ) [inline], [static]
```

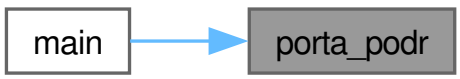
Definition at line 117 of file main.c.

```
00118 {
00119     return (volatile uint8_t *)k_porta_podr_addr;
00120 }
```

References k_porta_podr_addr.

Referenced by main().

Here is the caller graph for this function:



4.3.4.12 portb_pdr()

volatile uint8_t * portb_pdr (
void) [inline], [static]

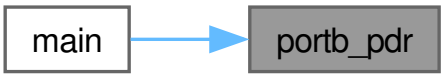
Definition at line 121 of file main.c.

```
00122 {
00123     return (volatile uint8_t *)k_portb_pdr_addr;
00124 }
```

References k_portb_pdr_addr.

Referenced by main().

Here is the caller graph for this function:



4.3.4.13 portb_podr()

volatile uint8_t * portb_podr (
void) [inline], [static]

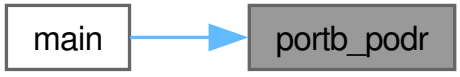
Definition at line 125 of file main.c.

```
00126 {
00127     return (volatile uint8_t *)k_portb_podr_addr;
00128 }
```

References k_portb_podr_addr.

Referenced by main().

Here is the caller graph for this function:



4.4 main.c

Go to the documentation of this file.

```
00001 /**
00002  * @file main.c
00003  * @brief RX72N multi-LED breathing cycle test: PA7, PB0, P71, P72, PB1, PB2.
00004  *
00005  * @details
00006  * Bare-metal test with no RTOS, no BSP, and no external clock configuration.
00007  * Six LEDs breathe sequentially using software PWM, then all flash together
00008  * to signal the end of one full cycle before repeating.
00009  *
00010  * Breathing sequence:
00011  *   PA7 -> PB0 -> P71 -> P72 -> PB1 -> PB2 -> [flash all x3] -> repeat
00012  *
00013  * Runs on the default internal LOCO clock at 240 kHz (OFS1 = 0xFFFFFFFF).
00014  * Software PWM timing is calibrated for this clock.
00015  *
00016  * Port register addresses from RX72N Hardware Manual (r01uh0824ej0111):
00017  *   PDR  = 0x0008C000 + port_index  (direction: 0=input, 1=output)
00018  *   PODR = 0x0008C020 + port_index  (output data)
00019  *   Port indices: 7=0x07, A=0x0A, B=0x0B
00020  *
00021  * After reset, PMR registers default to 0x00 (all GPIO) so no MPC
00022  * unlock is required.
00023  *
00024  * Built with -O0 so volatile delay loops are not optimised out.
00025  *
00026  * SPDX-License-Identifier: MIT
00027  * @copyright Copyright (c) 2026 Locked Inc.
00028  */
00029
00030 #include <stdint.h>
00031
00032 /* ---------------------------------------------------------------------------
00033  * Port register addresses.
00034  *
00035  * uintptr_t is mandatory for hardware addresses: on the RX72N target it maps
00036  * to uint32_t, but on an x86_64 unit-test host it is uint64_t.  Using
00037  * uint32_t directly would silently truncate on the 64-bit host.
00038  * --------------------------------------------------------------------------- */
00039 typedef enum : uintptr_t {
00040     k_port7_pdr_addr  = 0x0008C007U, /**< PORT7 direction register */
00041     k_port7_podr_addr = 0x0008C027U, /**< PORT7 output data register */
00042     k_porta_pdr_addr  = 0x0008C00AU, /**< PORTA direction register */
00043     k_porta_podr_addr = 0x0008C02AU, /**< PORTA output data register */
00044     k_portb_pdr_addr  = 0x0008C00BU, /**< PORTB direction register */
00045     k_portb_podr_addr = 0x0008C02BU, /**< PORTB output data register */
00046 } port_addr_t;
00047
00048 /* ---------------------------------------------------------------------------
00049  * Pin bit masks (one enum per port to avoid duplicate-value confusion).
00050  * --------------------------------------------------------------------------- */
00051 typedef enum : uint8_t {
00052     k_pin_pa7 = (uint8_t)(1U « 7U), /**< PORTA pin 7 */
00053 } porta_pins_t;
00054
00055 typedef enum : uint8_t {
00056     k_pin_p71 = (uint8_t)(1U « 1U), /**< PORT7 pin 1 */
00057     k_pin_p72 = (uint8_t)(1U « 2U), /**< PORT7 pin 2 */
00058 } port7_pins_t;
00059
00060 typedef enum : uint8_t {
00061     k_pin_pb0 = (uint8_t)(1U « 0U), /**< PORTB pin 0 */
00062     k_pin_pb1 = (uint8_t)(1U « 1U), /**< PORTB pin 1 */
00063     k_pin_pb2 = (uint8_t)(1U « 2U), /**< PORTB pin 2 */
00064 } portb_pins_t;
00065
00066 /* ---------------------------------------------------------------------------
00067  * Breathing and flash timing.
00068  *
00069  * Calibration (LOCO = 240 kHz, -O0):
00070  *   One delay() iteration = ~12 LOCO cycles = ~50 us.
00071  *   PWM period = k_max_bright iterations = 50 * 50 us = 2.5 ms (400 Hz).
00072  *   Each brightness step is held for k_reps_per_step PWM cycles:
00073  *     5 * 2.5 ms = 12.5 ms per step.
00074  *   Full breathe (in + out) = 2 * 51 steps * 12.5 ms ~= 1.3 s per LED.
00075  * --------------------------------------------------------------------------- */
00076 typedef enum : uint32_t {
00077     k_max_bright     = 50U,   /**< PWM duty-cycle upper bound (steps 0..50) */
00078     k_reps_per_step  = 5U,    /**< PWM cycles held at each brightness step */
00079     k_flash_count    = 3U,    /**< Completion flashes after one full cycle */
00080     k_flash_on_cnt   = 8000U, /**< Flash ON busy-wait count */
00081     k_flash_off_cnt  = 8000U, /**< Flash OFF busy-wait count */
00082 } breathe_params_t;
```

```
00083
00084 /* ---------------------------------------------------------------------
00085  * LED count
00086  * --------------------------------------------------------------------- */
00087 typedef enum : uint8_t {
00088     k_num_leds = 6U, /**< LEDs in the breathing sequence */
00089 } led_count_t;
00090
00091 /* ---------------------------------------------------------------------
00092  * LED descriptor
00093  * --------------------------------------------------------------------- */
00094 /**
00095  * @brief Binds a GPIO output register to a single pin's bit mask.
00096  */
00097 typedef struct {
00098     volatile uint8_t *podr; /**< Port output data register for this LED */
00099     uint8_t           mask; /**< Bit mask selecting this LED's pin */
00100 } led_t;
00101
00102 /* ---------------------------------------------------------------------
00103  * Inline port register accessors
00104  * --------------------------------------------------------------------- */
00105 static inline volatile uint8_t *port7_pdr(void)
00106 {
00107     return (volatile uint8_t *)k_port7_pdr_addr;
00108 }
00109 static inline volatile uint8_t *port7_podr(void)
00110 {
00111     return (volatile uint8_t *)k_port7_podr_addr;
00112 }
00113 static inline volatile uint8_t *porta_pdr(void)
00114 {
00115     return (volatile uint8_t *)k_porta_pdr_addr;
00116 }
00117 static inline volatile uint8_t *porta_podr(void)
00118 {
00119     return (volatile uint8_t *)k_porta_podr_addr;
00120 }
00121 static inline volatile uint8_t *portb_pdr(void)
00122 {
00123     return (volatile uint8_t *)k_portb_pdr_addr;
00124 }
00125 static inline volatile uint8_t *portb_podr(void)
00126 {
00127     return (volatile uint8_t *)k_portb_podr_addr;
00128 }
00129
00130 /* ---------------------------------------------------------------------
00131  * Busy-wait delay
00132  * --------------------------------------------------------------------- */
00133 /**
00134  * @brief Spin for approximately @p count loop iterations.
00135  * @param[in] count Iteration count. Each iteration is ~50 us at LOCO 240 kHz.
00136  */
00137 static void delay(uint32_t count)
00138 {
00139     volatile uint32_t i;
00140     for (i = 0U; i < count; i++) {
00141         __asm__ __volatile__("nop");
00142     }
00143 }
00144
00145 /* ---------------------------------------------------------------------
00146  * LED helpers
00147  * --------------------------------------------------------------------- */
00148 static inline void led_on(const led_t *led)
00149 {
00150     *led->podr |= led->mask;
00151 }
00152
00153 static inline void led_off(const led_t *led)
00154 {
00155     *led->podr &= (uint8_t)~led->mask;
00156 }
00157
00158 /* ---------------------------------------------------------------------
00159  * Breathing effect
00160  * --------------------------------------------------------------------- */
00161 /**
00162  * @brief Run one PWM ramp pass on @p led.
00163  *
00164  * @details
00165  * Steps @p on_time from 0 to @c k_max_bright (or vice versa depending on
00166  * @p invert), holding each level for @c k_reps_per_step PWM cycles.
00167  *
00168  * @param[in] led    LED to animate.
00169  * @param[in] invert When 0, ramp 0->max (fade in). When 1, ramp max->0 (fade out).
```

```
00170  */
00171 static void breathe_ramp(const led_t *led, uint32_t invert)
00172 {
00173     uint32_t b;
00174     uint32_t rep;
00175     uint32_t on_time;
00176     uint32_t off_time;
00177
00178     for (b = 0U; b <= k_max_bright; b++) {
00179         on_time  = (invert == 0U) ? b : (k_max_bright - b);
00180         off_time = k_max_bright - on_time;
00181         for (rep = 0U; rep < k_reps_per_step; rep++) {
00182             if (on_time > 0U) {
00183                 led_on(led);
00184                 delay(on_time);
00185             }
00186             led_off(led);
00187             if (off_time > 0U) {
00188                 delay(off_time);
00189             }
00190         }
00191     }
00192 }
00193
00194 /**
00195  * @brief Run one full breathe cycle (fade in then fade out) on @p led.
00196  * @param[in] led LED to animate.
00197  */
00198 static void breathe(const led_t *led)
00199 {
00200     breathe_ramp(led, 0U); /* in  */
00201     breathe_ramp(led, 1U); /* out */
00202     led_off(led);
00203 }
00204
00205 /* ---------------------------------------------------------------------------
00206  * Completion flash
00207  * --------------------------------------------------------------------------- */
00208 /**
00209  * @brief Flash all LEDs @c k_flash_count times to signal cycle completion.
00210  * @param[in] leds  LED descriptor array.
00211  * @param[in] count Number of entries in @p leds.
00212  */
00213 static void flash_all(const led_t *leds, uint8_t count)
00214 {
00215     uint32_t f;
00216     uint8_t  i;
00217
00218     for (f = 0U; f < k_flash_count; f++) {
00219         for (i = 0U; i < count; i++) {
00220             led_on(&leds[i]);
00221         }
00222         delay(k_flash_on_cnt);
00223
00224         for (i = 0U; i < count; i++) {
00225             led_off(&leds[i]);
00226         }
00227         delay(k_flash_off_cnt);
00228     }
00229 }
00230
00231 /* ---------------------------------------------------------------------------
00232  * Entry point
00233  * --------------------------------------------------------------------------- */
00234 /**
00235  * @brief Configure GPIO outputs and run the breathing cycle forever.
00236  * @return Never returns.
00237  */
00238 int main(void)
00239 {
00240     /* Set LED pins as outputs (PDR bit = 1). */
00241     *porta_pdr() |= (uint8_t)k_pin_pa7;
00242     *port7_pdr() |= (uint8_t)(k_pin_p71 | k_pin_p72);
00243     *portb_pdr() |= (uint8_t)(k_pin_pb0 | k_pin_pb1 | k_pin_pb2);
00244
00245     /* LED table -- order defines the breathing sequence. */
00246     const led_t leds[k_num_leds] = {
00247         { porta_podr(), (uint8_t)k_pin_pa7 },
00248         { portb_podr(), (uint8_t)k_pin_pb0 },
00249         { port7_podr(), (uint8_t)k_pin_p71 },
00250         { port7_podr(), (uint8_t)k_pin_p72 },
00251         { portb_podr(), (uint8_t)k_pin_pb1 },
00252         { portb_podr(), (uint8_t)k_pin_pb2 },
00253     };
00254
00255     for (;;) {
00256         uint8_t i;
```

```
00257         for (i = 0U; i < k_num_leds; i++) {
00258             breathe(&leds[i]);
00259         }
00260         flash_all(leds, k_num_leds);
00261     }
00262 }
```